

EXERCISE 1 — ELECTRONICS STORE INVENTORY SYSTEM

Create an Interface [2 Points]

Requirements:

- Create an interface `IElectronicDevice`. **(1 Point)**
- Define the following methods: **(1 Point)**

Return Type	Method Name	Description
void	<code>inputDevice()</code>	Input details of a device
void	<code>displayDevice()</code>	Display all details of the device

Create the `ElectronicDevice` Class [6 Points]

Requirements:

Create a class `ElectronicDevice` implementing `IElectronicDevice`.

Fields:

- `String name`
- `String brand`
- `int releaseYear`
- `double price`

Constructors:

- No-argument constructor **(0.5 Point)**
- Constructor with all fields **(0.5 Point)**

Getters and Setters:

- Standard getters and setters for all fields **(1 Point)**

Implement Methods:

- `inputDevice()` — use `BufferedReader` to input and validate: **(1 Point)**
 - `name, brand` ≥ 2 characters
 - `releaseYear` ≥ 2000 and $\leq$ current year
 - `price` > 0
- `displayDevice()` — nicely formatted output **(1 Point)**
- `isPremium()` — returns `true` if `price` > 1000.0 **(1 Point)**
- Inside `displayDevice()`, show: **(0.5 Point)**
 - `"Device '[Name]'` is a premium product."
 - OR `"Device '[Name]'` is standard."

- Call `isPremium()` and show the result in `displayDevice()` **(0.5 Point)**

Create the InventoryManager Class [5 Points]

Requirements:

Create class `InventoryManager`.

Manage:

- A list of `ElectronicDevice` objects via `ArrayList`

Methods:

Method Name	Description	Points
<code>addDevice()</code>	Add a new device (calls <code>inputDevice()</code>)	1
<code>showAllDevices()</code>	Display all devices	1
<code>findMostExpensive()</code>	Find and show the most expensive device	3

Create the Main Class [3 Points]

Requirements:

Create `Main` class.

In `main()`:

- Show menu: **(1.5 Points)**

```

=== ELECTRONICS STORE MENU ===
1. Add New Device
2. Show All Devices
3. Find Most Expensive Device
4. Exit
Your choice:

```

- Call appropriate methods from `InventoryManager` **(1.5 Points)**
- End the program on option 4

Exception Handling in `inputDevice()` [4 Points]

Requirements:

Improve input validation in `inputDevice()` using a custom exception.

Tasks:

1. Create a new exception class `InvalidDeviceDataException`, extending `Exception`. **(1 Point)**
2. In `inputDevice()`, throw the custom exception when any field is invalid. **(1 Point)**

3. Catch exceptions in a loop to allow retry until input is valid. **(1 Point)**
4. Show meaningful error messages for each validation failure. **(1 Point)**

EXERCISE 2 — COURSE MANAGEMENT SYSTEM

Abstract Class for Course

a) Create the abstract class Course

Tasks

Create an abstract class named Course.

The class should contain the following attributes:

- `courseId (String)`
- `courseName (String)`
- `instructor (String)`
- `startDate (String)`
- `maxStudents (int)`

Requirements

- Create a no-argument constructor.
- Create a parameterized constructor.
- Provide getter and setter methods for all attributes.
- Define the following abstract methods:

```
public abstract void input(BufferedReader reader) throws IOException;  
public abstract void display();  
public abstract String getCourseType();
```

CostCalculable Interface

a) Create the CostCalculable interface

Tasks

Create an interface named CostCalculable.

Requirements

Declare the following method:

```
double getCostMetric();
```

OnlineCourse Class

a) Create the OnlineCourse class

Tasks

Create a class named `OnlineCourse`, inheriting from the `Course` class and implementing the `CostCalculable` interface.

Add the following attributes:

- `platform (String)`
- `courseFee (double)`

Requirements

- Create a no-argument constructor.
- Create a full constructor (including inherited attributes and new attributes), using the `super` keyword.
- Provide getter and setter methods for the new attributes.
- Override the abstract methods inherited from `Course`:
 - `input()` must prompt the user to enter all `OnlineCourse` information using `BufferedReader`.
 - `display()` must display all information of an `OnlineCourse` object.
 - `getCourseType()` must return the string `"ONLINE"`.
- Implement the method inherited from `CostCalculable`:
 - `getCostMetric()` must return the course fee.

OfflineCourse Class

a) Create the OfflineCourse class

Tasks

Create a class named `OfflineCourse`, inheriting from the `Course` class and implementing the `CostCalculable` interface.

Add the following attributes:

- `roomName (String)`
- `hasLab (boolean)`

Requirements

- Create a no-argument constructor.
- Create a full constructor using the `super` keyword.
- Provide getter and setter methods for the new attributes.
- Override the abstract methods inherited from `Course`:

- `input()` must prompt the user to enter all `OfflineCourse` information using `BufferedReader`.
- `display()` must display all information of an `OfflineCourse` object.

b) Estimated Cost Calculation for Offline Courses

Tasks

In the `OfflineCourse` class, create the following method:

```
public double calculateEstimatedCost();
```

Cost calculation rules

- If `hasLab == true`: 200,000 VND per student
- If `hasLab == false`: 120,000 VND per student

Estimated cost = `maxStudents` × cost per student

- After calling the `display()` method, also display the estimated course cost.
- Implement the method inherited from `CostCalculable`:
 - `getCostMetric()` must return the estimated course cost calculated by `calculateEstimatedCost()`.
- Implement the remaining abstract method inherited from `Course`:
 - `getCourseType()` must return the string "OFFLINE".

Menu System and Program Testing

a) Data storage structure

Tasks

Use a single `ArrayList<Course>` to store all types of courses:

```
ArrayList<Course> courseList = new ArrayList<>();
```

Do not create separate lists for `OnlineCourse` or `OfflineCourse`.

b) Main menu

Tasks

Create a class named `CourseManagementTest` containing the `main` method to implement a menu-driven program.

The program must use `BufferedReader` for user input and provide the following options:

```
===== COURSE MANAGEMENT SYSTEM =====
```

1. Add an Online Course
2. Add an Offline Course
3. Display all Courses
4. Display Online Courses (sorted by cost in ascending order)

5. Display Offline Courses (sorted by cost in ascending order)
 6. Exit
- Your choice:

c) Menu functionality

Tasks

- When the user selects option (1), create an `OnlineCourse` object, call `input()`, and add it to `courseList`.
- When the user selects option (2), create an `OfflineCourse` object, call `input()`, and add it to `courseList`.
- When the user selects option (3), display all courses in `courseList` by calling `display()`.
- When the user selects option (4), filter `OnlineCourse` objects from `courseList`, sort them by `getCostMetric()` in ascending order, and display them.
- When the user selects option (5), filter `OfflineCourse` objects from `courseList`, sort them by `getCostMetric()` in ascending order, and display them.
- When the user selects option (6), exit the program.

EXERCISE 3 — MUSIC EVENT MANAGEMENT SYSTEM

In this assessment, you will build a simple Music Event Management System using OOP principles. The system will manage music events with attributes like event ID, event name, date, and number of attendees.

Abstract Class for MusicEvent

a) Create an Abstract Class: MusicEvent

Create an abstract class named `MusicEvent`. It should contain the following properties: - `eventID` (String) - `eventName` (String) - `date` (String) - `numberOfAttendees` (int)

Create two constructors: - One default constructor. - One parameterized constructor that initializes all properties.

Create getter and setter methods for the properties. Define two abstract methods: - `public abstract void input();` - `public abstract void display();`

b) Create the ConcertEvent Class

Create a class named `ConcertEvent`, which extends the `MusicEvent` class. Add the following properties: - `artistName` (String) - `genre` (String)

Create two constructors: - One with no arguments. - One with 6 arguments that sets values for both inherited and added properties. Use the `super` keyword to call the parent constructor.

Create getter and setter methods for the added properties. Implement the `input()` and `display()` methods inherited from the `MusicEvent` class: - `input()`: Prompts the user to

enter values for all properties of the ConcertEvent object. - `display()`: Prints all the properties of the ConcertEvent object.

FestivalEvent Class

a) Create the FestivalEvent Class

Create a class named FestivalEvent, which extends the MusicEvent class. Add the following properties: - `numberOfStages` (int) - `durationDays` (int)

Create two constructors: - One with no arguments. - One with 6 arguments that sets values for both inherited and added properties. Use the `super` keyword to call the parent constructor.

Create getter and setter methods for the added properties. Implement the `input()` and `display()` methods inherited from the MusicEvent class: - `input()`: Prompts the user to enter values for all properties of the FestivalEvent object. - `display()`: Prints all the properties of the FestivalEvent object.

b) Calculate Estimated Attendance

Create a new method `calculateEstimatedAttendance()` in the FestivalEvent class that calculates the estimated attendance of the festival based on the number of stages and duration in days. For example: - For each stage, estimate 1000 attendees per day. - Estimated Attendance = $\text{numberOfStages} \times \text{durationDays} \times 1000$

Example Calculation: if the festival has 3 stages and lasts 2 days, the estimated attendance would be 6000 ($3 \text{ stages} \times 2 \text{ days} \times 1000 \text{ attendees}$).

Create a method `calculateEstimatedAttendance()` that returns the estimated number of attendees based on the above criteria. After calling the `display()` method, also print the estimated attendance using this method.

Expected Result:

```
Event ID: EV001
Event Name: Summer Fest
Date: 2023-08-15
Number of Attendees: 5000
Number of Stages: 3
Duration Days: 2
Estimated Attendance: 6000 attendees
```

Testing and Menu System

a) Main Menu

Create a class MusicEventTest containing a main method that implements a menu-driven program. The menu should offer the following options:

Please select:

1. Input information for n Concert Events.

2. Input information for n Festival Events.
3. Display information of n Concert Events (Sorted by number of attendees descending).
4. Display information of n Festival Events (Sorted by duration days ascending).
5. Exit.

Your choice:

b) Input Information for Concert Events

When the user selects option 1, prompt the user to input the number of concert events and store their information in an array of `ConcertEvent` objects.

c) Input Information for Festival Events

When the user selects option 2, prompt the user to input the number of festival events and store their information in an array of `FestivalEvent` objects.

d) Display Sorted Concert Events

When the user selects option 3, display the list of `ConcertEvent` objects sorted by the number of attendees in descending order.

e) Display Sorted Festival Events

When the user selects option 4, display the list of `FestivalEvent` objects sorted by `durationDays` in ascending order.

f) Exit the Program

When the user selects option 5, exit the program.

Note: Ensure your program handles user inputs gracefully and includes necessary validations where appropriate.

EXERCISE 4 — EMPLOYEE MANAGEMENT SYSTEM

Create an Interface

Requirements: - Create an interface `IEmployee`. - The interface should have the following methods:

#	Return Type	Method Name	Description
1	void	input	Allow user to input the details of an employee.
2	void	display	Print all details of the employee.

Create the Employee Class

Requirements: - Create a class `Employee` that implements the `IEmployee` interface.

#	Data Type	Property Name	Description
1	int	id	Employee ID (must be unique).
2	String	name	Employee's full name.
3	String	department	Employee's department.
4	double	salary	Employee's salary (must be greater than 0).

Constructors: - No-argument constructor. - Constructor with all four properties as arguments.

Getters and Setters: Create getter and setter methods for all properties.

Implement Methods: - `input()`: Use `BufferedReader` to input employee details from the user. Validate: - Name: At least 5 characters. - Salary: Greater than 0. - `display()`: Print all employee properties in a readable format.

Implement a method named `calculateTax()` that calculates the employee's annual tax based on the following rules: - Tax = 10% of the salary if salary $\leq$ 50,000. - Tax = 20% of the salary if salary $>$ 50,000 and $\leq$ 100,000. - Tax = 30% of the salary if salary $>$ 100,000.

Output Format: When `calculateTax()` is called, display the tax amount in the format: "Employee [Name] has an annual tax of \$[TaxAmount]."

Example Input/Output:

Input:

Enter Employee Name: John Doe

Enter Employee Salary: 120000

Output:

Employee John Doe has an annual tax of \$36000.

Create EmployeeManagement Class

Requirements: - Create a class `EmployeeManagement`.

#	Access Modifier	Return Type	Method Name	Description
1	public	void	<code>addEmployee</code>	Allow the user to add 3 employees to an array.

#	Access Modifier	Return Type	Method Name	Description
2	public	void	showEmployees	Display all employees in the array.
3	public	void	sortEmployeesBySalary	Sort employees in ascending order of salary.

Create the Main Class

Requirements: - Create a class Main. - The main () method should do the following:

1. Display a menu for the user:

```
=== MENU ===
```

1. Add New Employee
2. Show All Employees
3. Sort Employees by Salary
4. Exit

Your choice:

2. Call the appropriate method from the EmployeeManagement class based on the user's choice.
3. Exit the program when the user chooses option 4.